

Direct Preference Optimization.

What if the reward model and the entire RL loop could be removed — and the policy trained *directly* on preferences with one supervised loss? DPO proves the KL-constrained RL problem has a closed-form optimum, inverts it, and watches the partition function cancel. The result is logistic regression on preferences.

THE EQUATION

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

Worked example. Concrete $\pi_{\theta}, \pi_{\text{ref}}$ probabilities for a winner and loser response: compute both log-ratios, the β -scaled margin, its sigmoid, and the loss — then sweep β .

Topic 3 of 3 in this module · companion to the course page · v1.0

Contents

1	The claim: skip the reward model entirely	2
2	Derivation: the closed-form optimal policy	2
3	The cancellation: $Z(x)$ disappears	3
4	Worked computation	4
5	Heatmap: implicit reward across the pair	4
6	Properties & gradient	5
7	Where this sits in the track	5
A	Reproducible (NumPy)	5

1 The claim: skip the reward model entirely

PPO (Topic 2) trains a reward model, then optimizes the policy against it with RL — four networks, unstable, expensive. DPO (Rafailov et al., 2023) makes a striking observation: the reward model and the RL loop are redundant. The KL-constrained reward objective has a *known* optimal policy, and that relationship can be inverted to express reward in terms of the policy itself. Substituting into the Bradley–Terry loss yields a single supervised loss on the original preference data.

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right] \quad (1)$$

DPO is “classification on preferences.” The quantity $\hat{r}_{\theta}(x, y) = \beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)}$ is an *implicit reward* baked into the policy. Eq. (1) is exactly the Topic 1 Bradley–Terry loss with this implicit reward in place of an external r_{ϕ} — so we run the same stable logistic-regression training, but the “reward model” *is* the language model.

2 Derivation: the closed-form optimal policy

Recall the per-prompt RLHF objective from Topic 2 (Eq. 1 there):

$$\max_{\pi} \mathbb{E}_{y \sim \pi} [r(x, y)] - \beta \text{KL}(\pi(\cdot | x) \| \pi_{\text{ref}}(\cdot | x)).$$

Writing the KL out and using $\text{KL}(\pi \parallel \pi_{\text{ref}}) = \mathbb{E}_{y \sim \pi} \log \frac{\pi}{\pi_{\text{ref}}}$, this equals $-\beta$ times

$$\mathbb{E}_{y \sim \pi} \left[\log \frac{\pi(y \mid x)}{\pi_{\text{ref}}(y \mid x)} - \frac{1}{\beta} r(x, y) \right] = \mathbb{E}_{y \sim \pi} \left[\log \frac{\pi(y \mid x)}{\pi_{\text{ref}}(y \mid x) e^{r(x, y)/\beta}} \right].$$

Define the normaliser (*partition function*) $Z(x) = \sum_y \pi_{\text{ref}}(y \mid x) e^{r(x, y)/\beta}$, which does not depend on π . Then the bracket becomes a KL divergence to a specific distribution:

$$= \mathbb{E}_{y \sim \pi} \left[\log \frac{\pi(y \mid x)}{\frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) e^{r(x, y)/\beta}} \right] - \log Z(x) = \text{KL}(\pi \parallel \pi^*) - \log Z(x).$$

KL is minimised (at 0) exactly when $\pi = \pi^*$, giving the unique optimum:

$$\pi^*(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y)\right) \quad (2)$$

2.1 Inverting to get the implicit reward

Take logs of Eq. (2) and solve for r :

$$\log \pi^*(y \mid x) = \log \pi_{\text{ref}}(y \mid x) + \frac{1}{\beta} r(x, y) - \log Z(x) \implies r(x, y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x).$$

Identifying the optimal policy with our trainable π_θ :

$$r(x, y) = \beta \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x) \quad (3)$$

The reward is recoverable from the policy alone — no separate network. The only nuisance is the intractable $Z(x)$.

3 The cancellation: $Z(x)$ disappears

Plug the implicit reward (3) into the Bradley–Terry preference probability from Topic 1 ($P(y_w \succ y_l) = \sigma(r_w - r_l)$). The difference of rewards for the *same prompt* x is:

$$r(x, y_w) - r(x, y_l) = \left[\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} + \beta \log Z(x) \right] - \left[\beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} + \beta \log Z(x) \right].$$

Because $Z(x)$ depends only on x — not on y — the two $\beta \log Z(x)$ terms are *identical* and cancel:

$$r(x, y_w) - r(x, y_l) = \beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}.$$

Substituting into $\mathcal{L}_{\text{RM}} = -\mathbb{E}[\log \sigma(r_w - r_l)]$ gives the DPO loss of Eq. (1) exactly. The intractable partition function never appears.

The single trick that makes DPO work: $Z(x)$ is a per-*prompt* constant, and Bradley–Terry only ever uses the *difference* of two rewards *at the same prompt*. The constant cancels, leaving a loss that depends only on π_θ , π_{ref} , and the preferred/dispreferred pair.

4 Worked computation

Treat each response’s probability as the full-sequence likelihood under each model. Take

$$\pi_\theta(y_w | x) = 0.42, \quad \pi_{\text{ref}}(y_w | x) = 0.30, \quad \pi_\theta(y_l | x) = 0.10, \quad \pi_{\text{ref}}(y_l | x) = 0.18, \quad \beta = 0.5.$$

Step 1 — log-ratios.

$$\log \frac{\pi_\theta(y_w)}{\pi_{\text{ref}}(y_w)} = \log \frac{0.42}{0.30} = \log 1.4 = 0.336472, \quad \log \frac{\pi_\theta(y_l)}{\pi_{\text{ref}}(y_l)} = \log \frac{0.10}{0.18} = \log 0.5\bar{5} = -0.587787.$$

The policy raised the winner’s likelihood and lowered the loser’s — the desired direction.

Step 2 — implicit rewards (the $\beta \log Z$ terms cancel in step 3, so we drop them):

$$\hat{r}_\theta(y_w) = \beta(0.336472) = 0.168236, \quad \hat{r}_\theta(y_l) = \beta(-0.587787) = -0.293893.$$

Step 3 — β -scaled margin.

$$m = \beta \left(\log \frac{\pi_\theta(y_w)}{\pi_{\text{ref}}(y_w)} - \log \frac{\pi_\theta(y_l)}{\pi_{\text{ref}}(y_l)} \right) = 0.5(0.336472 - (-0.587787)) = 0.5(0.924259) = 0.462129.$$

Step 4 — sigmoid and loss.

$$\sigma(0.462129) = \frac{1}{1 + e^{-0.462129}} = 0.613519, \quad \mathcal{L}_{\text{DPO}} = -\log(0.613519) = \boxed{0.488544}.$$

4.1 Sweeping β

The raw log-ratio margin is fixed at 0.924259; β rescales it before the sigmoid:

β	margin $m = \beta \cdot 0.924259$	$\sigma(m)$	$\mathcal{L}_{\text{DPO}} = -\log \sigma(m)$
0.1	0.092426	0.523090	0.648002
0.5	0.462129	0.613519	0.488544
1.0	0.924259	0.715909	0.334202

Larger β makes the implicit reward more confident in the (correct) ordering, lowering the loss. But β also controls how far π_θ is allowed to drift from π_{ref} — the same trade-off as the KL coefficient in Topic 2, now living inside one loss.

5 Heatmap: implicit reward across the pair

The β -scaled log-ratio $\hat{r}_\theta(y) = \beta \log \frac{\pi_\theta(y)}{\pi_{\text{ref}}(y)}$ is DPO’s implicit reward. Shaded by signed magnitude (deeper = higher reward), it shows the loss separating winner from loser. Values at $\beta = 0.5$.

	$\log \frac{\pi_\theta}{\pi_{\text{ref}}}$	$\hat{r}_\theta = \beta \log \frac{\pi_\theta}{\pi_{\text{ref}}}$
winner y_w	+0.336472	+0.168236
loser y_l	−0.587787	−0.293893

The margin between rows ($0.168236 - (-0.293893) = 0.462129$) is precisely the logit fed to the sigmoid.

6 Properties & gradient

The DPO gradient (analogous to Topic 1) is

$$\nabla_\theta \mathcal{L}_{\text{DPO}} = -\beta (1 - \sigma(m)) \left[\nabla_\theta \log \pi_\theta(y_w | x) - \nabla_\theta \log \pi_\theta(y_l | x) \right],$$

with m the margin above. The weight $1 - \sigma(m)$ up-weights pairs the model currently gets wrong, and the update raises $\log \pi_\theta(y_w)$ while lowering $\log \pi_\theta(y_l)$.

1. **No reward model, no RL.** Just two networks (policy + frozen reference) and one loss.
2. **Stable.** Supervised cross-entropy-like training; no sampling, no value function, no clipping.
3. **Same optimum.** Under the Bradley–Terry assumption, DPO targets the same policy PPO does.
4. β is still the **KL knob**, now implicit in the loss scale.

DPO is *not* free of assumptions. It still needs (i) a good reference π_{ref} (usually the SFT model) — the log-ratios are meaningless otherwise — and (ii) preference data that is *on-distribution* for that reference. Off-distribution or noisy preferences can drive π_θ to reduce the loss by simply suppressing $\pi_\theta(y_l)$ toward zero, degrading the model’s overall likelihood. In practice DPO can over-optimize and lose diversity if β is too small or the data is stale.

7 Where this sits in the track

	Reward model?	RL loop?	Networks
Topic 2 (PPO)	yes (explicit r_ϕ)	yes	4
Topic 3 (DPO)	implicit (in π_θ)	no	2

This closes Module 2.3: from preferences $\rightarrow$ reward (T1) $\rightarrow$ RL with a leash (T2) $\rightarrow$ a single supervised loss that does both at once (T3).

A Reproducible (NumPy)

Inputs: $\pi_\theta(y_w) = 0.42$, $\pi_{\text{ref}}(y_w) = 0.30$, $\pi_\theta(y_l) = 0.10$, $\pi_{\text{ref}}(y_l) = 0.18$, $\beta = 0.5$. Outputs: log-ratios 0.336472, −0.587787; margin 0.462129; $\sigma = 0.613519$; loss 0.488544.

```
import numpy as np
sig = lambda z: 1/(1+np.exp(-z))

pt_w, pr_w = 0.42, 0.30
pt_l, pr_l = 0.10, 0.18
beta = 0.5

lr_w = np.log(pt_w / pr_w)          # 0.3364722366
lr_l = np.log(pt_l / pr_l)          # -0.5877866649
margin = beta * (lr_w - lr_l)        # 0.4621294508
loss = -np.log(sig(margin))          # 0.4885436880
print(lr_w, lr_l, margin, sig(margin), loss)

for b in (0.1, 0.5, 1.0):            # beta sweep
    m = b * (lr_w - lr_l)
    print(b, m, sig(m), -np.log(sig(m)))
# 0.1 -> loss 0.648002 ; 0.5 -> 0.488544 ; 1.0 -> 0.334202
```

Marevlo Research · Transformer Track · Module 2.3 · Topic 3 of 3.